CHIP-8 主循环中的 Rust 语法

从能运行的代码理解所有权、错误处理与时间调度

对应项目：Chip8_rust/src/main.rs 2026-10-13

这份讲义围绕刚接好的主循环、Platform 和 Chip::cycle 展开。目标是读懂代码中出现的 Rust 语法，而不是逐条讲解 CHIP-8 指令的语义。

1 先看整体：谁负责什么

组成	职责
Chip	保存内存、寄存器、按键和画面；执行一条指令或更新定时器。
Platform	管理 SDL 窗口、事件和纹理更新。
main	加载 ROM，创建对象，按时间安排指令、定时器和绘制。

这与 C++ 的“类加主函数”在职责上类似。Rust 用 struct 存字段，用 impl 定义关联函数和方法：

```
struct Chip { pc: u16, /* other fields */ }

impl Chip {
    fn new() -> Self { /* construct a Chip */ }
    fn cycle(&mut self) -> Result<(), String> { /* one instruction */ }
}
```

Self 表示当前 impl 对应的类型，这里就是 Chip。Chip::new() 是关联函数；带 &mut self 的 cycle() 是会修改对象的方法。

2 函数返回值与 Result

主函数的签名是：

```
pub fn main() -> Result<(), Box<dyn std::error::Error>> {
    // ...
    Ok(())
}
```

- -> 后面是返回类型。
- Result<T, E> 表示成功时返回 Ok(T)，失败时返回 Err(E)。
- () 是单元类型：成功时无需附带数据，因此返回 Ok(())。
- Box<dyn Error> 是装在堆上的错误 trait 对象，使主函数可以接收多种错误类型。

加载 ROM、创建 SDL 对象、执行指令时都可能失败，所以主函数使用?：

```
chip.load_rom(&rom_path)?;
let mut platform = Platform::new(10)?;
chip.cycle()?;
platform.update(&mut texture, &chip.video)?;
```

? 对 Ok(value) 取出 value，遇到 Err(error) 则立即从当前函数返回错误。它让主循环只保留正常路径。? 需要所在函数能够返回兼容的 Result。

3 let、mut 与作用域

```
let mut chip = Chip::new();
let instruction_interval = Duration::from_millis(2);
let mut last_time = Instant::now();
```

变量默认不能重新赋值或通过它修改内部状态。mut 表示需要修改：chip.cycle() 要修改 Chip；每轮循环也会更新 last_time。时间间隔不变，所以 instruction_interval 不需要 mut。

变量只在所在的花括号作用域内有效。循环体中的 let now = Instant::now(); 每轮都会创建一个新的 now；last_time 定义在循环外，能跨轮次保留上次时间。

4 Option 与闭包：可选的 ROM 路径

```
let rom_path = std::env::args()
    .nth(1)
    .unwrap_or_else(|| "test_opcode.ch8".to_owned());
```

nth(1) 尝试取第一个用户参数，返回 Option<String>：有参数为 Some(path)，没有则为 None。unwrap_or_else 在 None 时调用后面的闭包，生成默认路径。

|| ... 是无参数闭包。to_owned() 把字符串字面量转成拥有数据的 String，与命令行参数的类型一致。

Option 表示“可能没有值”；Result 表示“操作可能失败”。两者都有匹配、转换和提取值的方法，但含义不同。

---

5 借用：&T 与 &mut T

主循环把 Chip 的不同字段交给 SDL：

```
platform.process_input(&mut chip.keypad);
platform.update(&mut texture, &chip.video)?;
```

&mut chip.keypad 是可变借用，允许 process_input 把按键状态改成 0 或 1。&chip.video 是不可变借用，update 只读取画面。&mut texture 允许 SDL 把新像素写进纹理。这些借用只覆盖调用期间，调用结束后主循环仍可继续使用 chip。

对应的方法参数通常写成：

```
fn process_input(&mut self, keys: &mut [u8; 16]) -> bool
fn update(&mut self, texture: &mut Texture<'_>,
          screen: &[u32; 64 * 32]) -> Result<(), String>
```

[u8; 16] 是固定长度数组；&mut [u8; 16] 是这个数组的可变引用。Texture<'_> 中的生命周期占位符说明纹理依赖 TextureCreator；因此主函数先保存 texture_creator，再创建 texture，让创建者至少活到纹理使用完毕。

6 循环与条件：安排每一帧

```
loop {
    if platform.process_input(&mut chip.keypad) {
        break;
    }
}
```

loop 是无限循环；break 离开最近的一层循环。这里关闭窗口或按 Escape 时，process_input 返回 true，主循环便结束。

if drew_frame { ... } 只在这一轮执行过指令时更新画面。布尔值直接作为条件，无须写成 drew_frame == true。

7 Instant、Duration 与累加器

```
let instruction_interval = Duration::from_millis(2);
let timer_interval = Duration::from_secs_f64(1.0 / 60.0);
let mut instruction_elapsed = Duration::ZERO;
let mut timer_elapsed = Duration::ZERO;
let mut last_time = Instant::now();

loop {
    let now = Instant::now();
    let elapsed = now.duration_since(last_time);
    last_time = now;
    instruction_elapsed += elapsed;
    timer_elapsed += elapsed;

    while instruction_elapsed >= instruction_interval {
        chip.cycle()?;
        instruction_elapsed -= instruction_interval;
    }
    while timer_elapsed >= timer_interval {
        chip.tick_timers();
        timer_elapsed -= timer_interval;
    }
}
```

`Instant` 表示单调时钟上的一个时刻；`Duration` 表示一段时长。`duration_since` 算出本轮距离上一轮经过多久。代码把同一段时间分别累计给 CPU 指令和计时器，故两个速度互不绑定。

这里 `2 ms` 对应大约每秒 500 条指令；`1/60 s` 对应 CHIP-8 定时器的 60 Hz。用 `while` 而不是 `if`，是为了在一轮耗时超过一个间隔时补执行多次。末尾的 `sleep(Duration::from_millis(1))` 降低空转占用，执行频率仍由 `Instant` 测得的时间决定。

如果某轮耗时 5 ms，指令间隔是 2 ms，`while` 会执行两条指令并留下约 1 ms；单次 `if` 只能执行一条，模拟器会逐渐落后。

---

8 match 与模式匹配

SDL 事件用枚举表示。match 不只比较数值，还能拆出事件携带的数据：

```
match event {
    Event::Quit { .. }
    | Event::KeyDown { keycode: Some(Keycode::Escape), .. }
        => return true,
    Event::KeyDown { keycode: Some(key), .. } => {
        if let Some(index) = Self::key_index(key) {
            keys[index] = 1;
        }
    }
    _ => {}
}
```

- | 表示两个模式共用一个处理分支。
- .. 忽略事件中其他字段。
- Some(key) 从 Option<Keycode> 中取出按键。
- if let Some(index) = ... 只在映射成功时处理索引。
- _ 是兜底模式；Self::key_index 调用当前 impl 中的关联函数。

指令分发也使用 match：

```
match self.opcode & 0xF000 {
    0x1000 => self.op_1nnn(),
    0x5000 if self.opcode & 0x000F == 0 => self.op_5xy0(),
    0xD000 => self.op_dxyn().map_err(str::to_owned)?,
    _ => return Err(format!("Unknown opcode: {:#06X}", self.opcode)),
}
```

0x5000 if ... 是带守卫的匹配分支：高四位等于 5 且最低四位等于 0 时才执行。注意这里的 & 是**按位与**；&chip.video 中的 & 才是**取引用**，位置不同，含义不同。

9 安全取指：切片、转换与整数运算

```
let pc = usize::from(self.pc);
let instruction = self.memory.get(pc..pc + 2)
    .ok_or_else(|| format!("Bad PC: {:#05X}", self.pc))?;
self.opcode = u16::from_be_bytes([instruction[0], instruction[1]]);
self.pc = self.pc.wrapping_add(2);
```

数组下标使用 usize，所以把 u16 的程序计数器转换为 usize。pc..pc + 2 是左闭右开的范围，只取两个字节。get 在越界时返回 None；ok_or_else 把它变成错误，再由? 返回。

CHIP-8 指令把高字节放在前面。代码里的 from_be_bytes 把两个字节组合成 u16。

wrapping_add(2) 在整数上限处按固定位宽回绕；普通加法在调试构建中溢出可能触发 panic。下次取指仍会检查程序计数器是否落在内存内。

10 迭代器与像素字节

```
let pixels: Vec<u8> = screen.iter()
    .flat_map(|pixel| pixel.to_ne_bytes())
    .collect();

texture.update(None, &pixels, 64 * 4)?;
```

iter() 逐个借用 u32 像素；|pixel| ... 是接收一个像素的闭包；to_ne_bytes() 把一个 u32 转成 4 个本机字节；flat_map 将这些小数组接成一个字节序列；collect() 收集为 Vec<u8>。

texture.update 接受字节切片 &pixels。最后一个参数是每行字节数（pitch）：64 像素乘每像素 4 字节，得到 256。当前画面只用全 0 和全 1 的 u32，因此这两种像素的颜色字节不受字节序影响。

11 记住这四个小差别

1. 0..x 不包含 x；0..=x 包含 x。FX55 要复制 V0 到 Vx，所以用后者。
2. &T 只读借用；&mut T 可变借用；& 在位运算表达式中又可表示按位与。
3. Option<T> 处理“可能不存在”；Result<T,E> 处理“可能出错”。
4. sleep(1 ms) 不是精确的 1 ms 定时器；实际模拟速度由测得的 elapsed 和两个时间累加器控制。

自测

1. 当 process_input 返回 true，break 会离开哪一层循环?
2. 为什么 texture_creator 要在 texture 使用期间保持有效?
3. instruction_elapsed = 5 ms、间隔 2 ms 时，本轮执行几条指令?
4. memory.get(pc..pc + 2) 越界时先得到 None 还是 Err?

答案：1. 主循环；2. 纹理借用创建者；3. 两条，余约 1 ms；4. 先得到 None，再经 ok_or_else 变成 Err。